DISCIPLINA DE QUALIDADE DE SOFTWARE
Especificação de Projeto em Grupo, Grupo 01

Frameworks de Teste Unitário

Estudo comparativo (benchmarking) entre Japa, Jest e Vitest aplicado à API *girls-chat-api* (AdonisJS 5 + TypeScript)

Equipe

Ana Alícia

Emanuele Maria

Fábio Henrique

Joserlan Gonçalves

Joel Fentes

Professora responsável: Profa. Carol Melo

Novembro de 2026

Sumário

1. Introdução
2. Fundamentação
3. Metodologia
4. Resultados quantitativos
5. Resultados qualitativos
6. Tabela comparativa consolidada
7. Análise crítica e recomendações
8. Limitações do estudo
9. Conclusão

Referências e Apêndice

1. Introdução

O teste unitário verifica isoladamente o comportamento de pequenas unidades de código (funções, métodos e classes), permitindo detectar defeitos cedo e sustentar refatorações seguras. A escolha do framework de teste afeta a produtividade da equipe, a legibilidade dos testes, a manutenção e a integração com pipelines de integração contínua.

Este relatório compara três frameworks de teste unitário do ecossistema JavaScript/TypeScript (**Japa**, **Jest** e **Vitest**) a partir da implementação do **mesmo conjunto de 12 casos de teste** sobre o mesmo sistema-alvo: os serviços de domínio da *girls-chat-api*, uma API de chat em AdonisJS 5. A escolha permanece em uma única linguagem para uma comparação mais justa. O Japa é o runner oficial do AdonisJS, o Jest é o padrão de mercado e o Vitest é a alternativa moderna baseada no Vite.

2. Fundamentação

2.1 Teste unitário, padrão AAA e princípios FIRST

O padrão **AAA (Arrange–Act–Assert)** organiza cada teste em três blocos: preparar o cenário (dados e dublês), executar a ação sob teste e verificar o resultado. Todos os testes deste estudo seguem o AAA, separando visualmente os blocos por linhas em branco.

Os princípios **FIRST** descrevem bons testes unitários: **Fast** (rápidos, em milissegundos), **Independent** (sem dependência de ordem ou de estado compartilhado), **Repeatable** (mesmo resultado em qualquer ambiente), **Self-validating** (passam ou falham sem inspeção manual) e **Timely** (escritos junto com o código). Para atender ao *Fast*, ao *Independent* e ao *Repeatable*, os testes deste estudo **não acessam banco de dados**: os models do ORM Lucid são substituídos por dublês, e os dublês são restaurados ou resetados antes de cada teste.

2.2 Características declaradas pelos frameworks

Recurso	Japa 2 (@japa/runner)	Jest 30	Vitest 5
Mocking	Não embutido; recomenda bibliotecas externas (ex.: Sinon)	jest.fn, jest.mock (módulos), spyOn, timers falsos	vi.fn, vi.mock (módulos), spyOn, timers falsos
Parametrização	`test(...).with([...])` (datasets)	`test.each`	`test.each` / `test.for`
Relatórios	spec reporter; plugins de terceiros	default, verbose, JUnit (pacote externo), personalizados	default, verbose, JUnit, JSON, HTML, github-actions
Cobertura	Externa (c8/nyc)	Embutida (Istanbul ou V8)	Pacote oficial (@vitest/coverage-v8 ou istanbul)
TypeScript	Via require-ts do Adonis	ts-jest, Babel ou SWC	Nativo (esbuild/Vite)
Diferencial	Integração com o Adonis (IoC, cliente HTTP, banco)	Ecossistema, snapshots, maturidade	Velocidade, API compatível com Jest, ESM

3. Metodologia

3.1 Sistema-alvo

Foram escolhidos os serviços `ChatsService` e `UserService` da API, que concentram regras de negócio não triviais: criação de chat apenas entre usuários existentes e sem chat duplicado (em qualquer ordem dos participantes), busca do chat em comum entre dois usuários, atualização da última mensagem, controle de participantes (sem duplicidade) e cadastro de usuário com validação de e-mail, unicidade de e-mail e senha armazenada como hash MD5. O código de produção **não foi alterado**.

3.2 Casos de teste

Os 12 casos abaixo (5 de sucesso e 7 de falha/exceção) foram implementados com o mesmo nome, os mesmos dados e as mesmas asserções nos três frameworks. Os dublês (query builder falso, usuários, chats e participantes) ficam em um módulo independente de framework (`tests/frameworks/shared/fakes.ts`), e cada framework usa o próprio mecanismo para injetá-los: `sinon.stub` no Japa, `jest.mock` no Jest e `vi.mock` no Vitest.

#	Caso de teste	Tipo
1	ChatsService: cria chat entre dois usuários com host e dois participantes	Sucesso
2	ChatsService: falha ao criar chat sem os IDs obrigatórios	Exceção
3	ChatsService: falha ao criar chat quando o host não existe	Exceção
4	ChatsService: falha ao criar chat duplicado entre os mesmos usuários	Exceção
5	ChatsService: encontra o chat em comum entre dois usuários	Sucesso
6	ChatsService: atualiza a última mensagem e a data do chat	Sucesso
7	ChatsService: falha ao atualizar a última mensagem de chat inexistente	Exceção
8	ChatsService: falha ao adicionar usuário que já participa do chat	Exceção
9	ChatsService: remove participante existente do chat	Sucesso
10	UserService: cria usuário armazenando a senha com hash MD5	Sucesso
11	UserService: falha ao criar usuário com e-mail inválido	Exceção
12	UserService: falha ao criar usuário com e-mail já cadastrado	Exceção

Os casos de exceção também verificam efeitos colaterais: nos testes 2, 4, 8, 11 e 12 confere-se que nenhum registro foi criado ou consultado indevidamente. Organização no repositório: `tests/frameworks/japa`, `tests/frameworks/jest` e `tests/frameworks/vitest` (branch `benchmark/frameworks-teste-unitario`).

3.3 Métricas e método de medição

- **Tempo de execução:** o script `scripts/benchmark/run-benchmark.js` executa, para cada framework, 1 rodada de aquecimento (descartada) e **5 execuções consecutivas** da suíte. Registra o *tempo de parede* do processo (do spawn ao término, com `process.hrtime`) e o *tempo interno* relatado pelo framework. O script aborta se alguma execução não rodar exatamente 12 testes.
- **Volume de código:** o script `scripts/benchmark/count-loc.js` conta as linhas excluindo brancos e comentários.
- **Cobertura:** provedor V8 nos três (c8 no Japa, `coverageProvider: 'v8'` no Jest e @vitest/coverage-v8 no Vitest), sobre `ChatsService.ts` e `UserService.ts`.
- **Esforço de configuração:** número de passos e de linhas de configuração para sair de um projeto sem testes até rodar testes com cobertura.
- **Outros indicadores:** número de dependências transitivas e tamanho em disco de uma instalação limpa dos pacotes de cada framework.

Ambientes de medição. O tempo foi medido em dois ambientes, com o mesmo script e o mesmo código: **Medição 1**, VM/contêiner Linux em nuvem (onde o estudo foi montado), e **Medição 2**, Windows pessoal (testes via CMD). Os demais indicadores (linhas, configuração, dependências e notas) não dependem do ambiente. A cobertura foi medida no Linux para os três frameworks e repetida no Windows para o Vitest.

Medição 1: Linux 6.18.44-fc-v236 (x64), Intel(R) Xeon(R) Processor @ 2.30GHz (4 vCPUs), 15.7 GB de RAM, Node.js v22.23.4. Medição 2: Windows pessoal (testes via CMD); o log registra o tempo de parede de cada execução e só a média do tempo interno. Versões: @japa/runner 2.5.1 + Sinon 22.1.0 + c8 12.0.0; Jest 30.5.2 + ts-jest 29.4.15; Vitest 5.0.5; TypeScript 4.6.4. Medição em 05/11/2026.

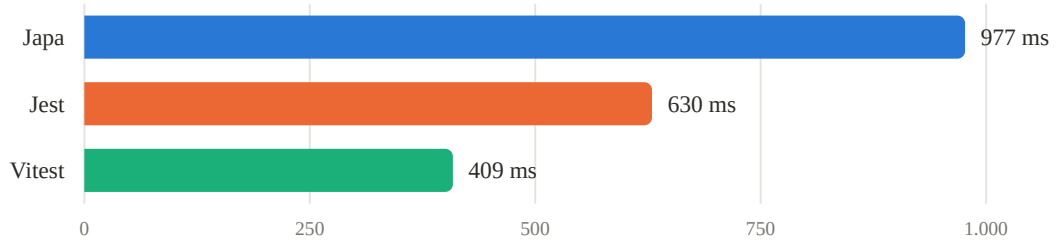
4. Resultados quantitativos

4.1 Tempo de execução

Medição 1: VM/contêiner Linux em nuvem

Framework	#1	#2	#3	#4	#5	Média (ms)	Desvio (ms)	Interno (ms)
Japa	1.018,6	1.070,3	920,2	958,7	915,7	**976,7**	66,6	38,6
Jest	544,5	606,1	669,0	677,1	650,9	**629,5**	54,9	342,8
Vitest	483,0	377,2	370,7	414,7	397,9	**408,7**	45,0	168,6

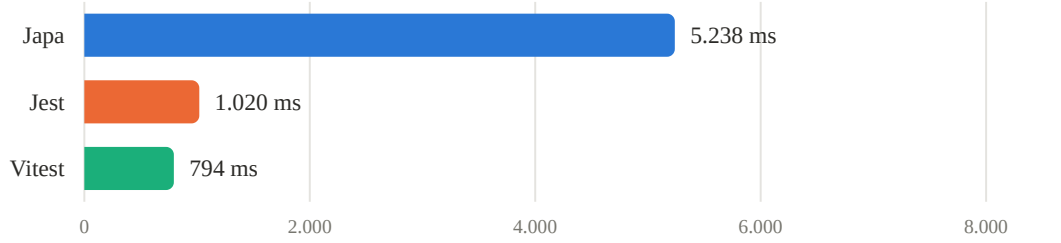
Medição 1 (Linux): tempo médio de parede, 12 testes (menor é melhor)



Medição 2: Windows pessoal (testes via CMD)

Framework	#1	#2	#3	#4	#5	Média (ms)	Desvio (ms)	Interno (ms)
Japa	5.012,0	5.100,0	5.061,0	5.374,0	5.645,0	**5.238,3**	267,3	55,2
Jest	1.046,0	1.000,0	1.050,0	1.012,0	991,0	**1.019,8**	27,1	610,8
Vitest	772,0	807,0	793,0	826,0	772,0	**794,0**	23,3	303,2

Medição 2 (Windows): tempo médio de parede, 12 testes (menor é melhor)



No Windows, a coluna "Interno" é a média impressa pelo script; o tempo interno de cada execução não foi registrado.

O **Vitest** foi o mais rápido de ponta a ponta (408,7 ms), seguido do **Jest** (629,5 ms) e do **Japa** (976,7 ms). O tempo interno, porém, mostra outra leitura: o Japa executa os testes em apenas 38,6 ms, mas antes precisa inicializar a aplicação Adonis inteira (providers, rotas, Firebase), e esse custo fixo

domina o tempo total. No Jest, a maior parte do tempo vai para a transformação TypeScript feita pelo ts-jest; o Vitest usa esbuild, que é mais rápido.

Na Medição 2 (Windows), todos os tempos foram maiores (Vitest 794,0 ms, Jest 1.019,8 ms e Japa 5.238,3 ms), mas **a ordem foi a mesma**: Vitest < Jest < Japa. Os valores absolutos de um ambiente não devem ser comparados com os do outro, porque as máquinas são diferentes; o resultado que se repete é a classificação entre os frameworks.

4.2 Volume de código

Framework	Linhas de teste	+ dublês compartilhados	Linhas de configuração
Japa	130	170	71
Jest	125	165	10
Vitest	123	163	15

Os volumes de teste são praticamente iguais (123 a 130 linhas), porque a lógica dos testes é a mesma. O Japa tem algumas linhas a mais por precisar criar e restaurar manualmente os stubs do Sinon, enquanto o Jest e o Vitest declaram os mocks de módulo de forma concisa. A diferença real está na configuração: no Japa são 71 linhas (arquivos de bootstrap gerados pelo Adonis e o registro da suíte), contra 10 no Jest e 15 no Vitest.

4.3 Cobertura de código

Framework	Arquivo	Linhas	Instruções	Ramos	Funções
Japa	**Total**	45,0%	45,0%	72,7%	40,0%
Japa	ChatsService.ts	52,6%	52,6%	67,9%	62,5%
Japa	UserService.ts	31,2%	31,2%	100,0%	14,3%
Jest	**Total**	45,0%	45,0%	72,7%	40,0%
Jest	ChatsService.ts	52,6%	52,6%	67,9%	62,5%
Jest	UserService.ts	31,2%	31,2%	100,0%	14,3%
Vitest	**Total**	37,3%	38,0%	39,2%	32,1%
Vitest	ChatsService.ts	52,5%	52,9%	52,3%	38,1%
Vitest	UserService.ts	14,9%	14,9%	22,9%	14,3%

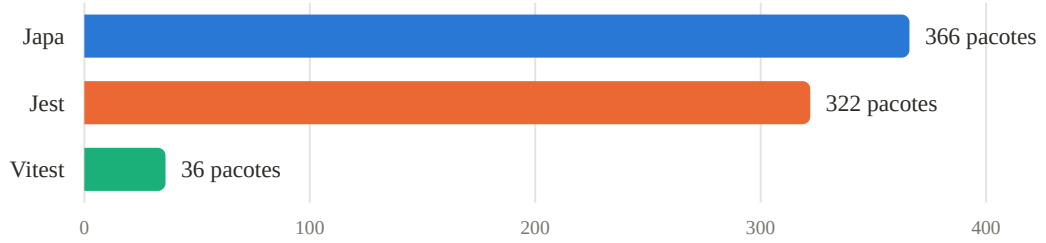
Os mesmos 12 testes exercitam exatamente as mesmas linhas, mas as ferramentas contam de forma diferente. O Japa (c8) e o Jest (V8) convertem a cobertura linha a linha a partir dos *source maps* do compilador TypeScript e dão números idênticos (45,0% de linhas). O Vitest usa remapeamento guiado pela AST, que desconsidera linhas sem instruções (chaves, declarações) e é mais rigoroso, chegando a 37,3%. Repetida no Windows (Medição 2), a cobertura do Vitest deu o mesmo valor: 37,3% de linhas e 39,2% de ramos. Os percentuais são moderados porque os 12 testes focam as regras

de negócio de criação e manutenção de chats e de cadastro; métodos de consulta com *preload* e o upload de foto (Firebase) ficaram fora do escopo.

4.4 Esforço de configuração e dependências

Framework	Passos	Pacotes instalados	Dependências transitivas	Tamanho (MB)
Japa	5	@japa/runner@2.5.1, @japa/preset-adonis@1.2.0, sinon@22.1.0, c8@12.0.0	366	67
Jest	2	jest@30.5.2, ts-jest@29.4.15, @types/jest@30	322	63
Vitest	2	vitest@5.0.5, @vitest/coverage-v8@5.0.5	36	12

Dependências transitivas instaladas (menor é melhor)



O Japa exige 5 passos: além do runner, foi preciso adicionar o Sinon para os dublês e o c8 para a cobertura, já que nenhum dos dois vem embutido. O Jest e o Vitest exigem 2 passos (instalar e criar o arquivo de configuração). O Vitest instala 36 pacotes (12 MB), cerca de 9 vezes menos que o Jest.

5. Resultados qualitativos

Escala: 1 = muito ruim, 5 = excelente. As notas foram atribuídas a partir da experiência de implementação e das saídas reais de falha (Apêndice A).

Critério	Japa	Jest	Vitest
Curva de aprendizado	3	4	5
Documentação oficial	3	5	4
Mensagens de erro	3	5	4
Integração com IDE e CI/CD	3	5	5
Mocking embutido	2	5	5
Suporte a TypeScript/ESM	4	3	5
Integração com AdonisJS	5	2	2
Ecossistema e comunidade	2	5	4
Média	**3,13**	**4,25**	**4,25**

Justificativas

Curva de aprendizado

- **Japa (3):** API pequena (test.group/test/assert), mas pouco material fora da documentação do Adonis; exige aprender Sinon à parte para dublês, e assert.rejects(fn, 'texto') exige mensagem exata (armadilha encontrada na prática).
- **Jest (4):** API expect amplamente conhecida e muito material didático; o atrito está na configuração TypeScript (ts-jest, moduleNameMapper para os aliases App/*).
- **Vitest (5):** Mesma API do Jest (describe/test/expect/vi), TypeScript sem transpilador extra e configuração mínima; quem conhece Jest migra trocando jest. por vi.

Documentação oficial

- **Japa (3):** Clara e objetiva, porém enxuta; a versão 2 (usada pelo Adonis 5) convive com a v3/v4, o que confunde buscas. Poucos exemplos de mocking.
- **Jest (5):** Extensa, madura, com guias de mocking, temporizadores, snapshots e referência completa de matchers.
- **Vitest (4):** Completa e bem organizada, com guia de migração do Jest; mudanças frequentes entre versões maiores exigem atenção à versão.

Mensagens de erro

- **Japa (3):** Diff legível em deepEqual e trecho do código do teste, mas truncou a mensagem recebida em rejects ('Chat já existe …') e exibiu um aviso confuso sobre tipos; asserções sobre stubs do Sinon (isFalse(stub.called)) não mostram contexto das chamadas.
- **Jest (5):** Mostra a substring esperada e a mensagem recebida completas, o trecho do serviço onde o erro foi lançado, diff de argumentos do mock e 'Number of calls'.
- **Vitest (4):** Expected/Received completos, diff dos argumentos do vi.fn() e número de chamadas; a linha do código-fonte exibida é truncada ('Chat d…').

Integração com IDE e CI/CD

- **Japa (3):** Roda em qualquer CI via 'node ace test'; integração com IDEs é limitada (sem runner nativo no WebStorm/VS Code para a v2).
- **Jest (5):** Suporte nativo no WebStorm, extensão oficial para VS Code, reporters JUnit e cobertura integrada; padrão de mercado em pipelines.
- **Vitest (5):** Extensão oficial para VS Code, suporte no WebStorm, modo watch rápido e reporter 'github-actions' embutido.

Mocking embutido

- **Japa (2):** Não possui: foi preciso instalar e configurar o Sinon e restaurar os stubs manualmente após cada teste.
- **Jest (5):** jest.mock/jest.fn/spyOn com mock de módulo inteiro e matchers de chamadas (toHaveBeenCalledWith).
- **Vitest (5):** vi.mock/vi.fn/spyOn equivalentes ao Jest, com hoisting automático de vi.mock.

Suporte a TypeScript/ESM

- **Japa (4):** Transparente dentro do Adonis (require-ts compila sob demanda), mas acoplado ao boot da aplicação.
- **Jest (3):** Precisa do ts-jest (ou Babel/SWC); ESM ainda é experimental e houve aviso de opção obsoleta durante a configuração.
- **Vitest (5):** TypeScript e ESM nativos via Vite/esbuild, sem transformadores adicionais.

Integração com AdonisJS

- **Japa (5):** Runner oficial do Adonis: sobe o container IoC, permite testes HTTP (api-client) e transações de banco; os models reais podem ser usados.
- **Jest (2):** Não inicializa o IoC do Adonis; módulos @ioc:* precisam ser mockados, o que restringe o uso a testes puramente unitários.
- **Vitest (2):** Mesma limitação do Jest: os models que importam @ioc:Adonis/Lucid/Orm precisam ser substituídos por mocks.

Ecossistema e comunidade

- **Japa (2):** Comunidade pequena, praticamente restrita ao AdonisJS; poucos plugins de terceiros.
- **Jest (5):** O mais adotado do ecossistema JavaScript; grande quantidade de plugins, exemplos e respostas na comunidade.
- **Vitest (4):** Crescimento acelerado e compatível com boa parte dos plugins do Jest, mas ainda menor que o Jest.

6. Tabela comparativa consolidada

↓ menor é melhor · ↑ maior é melhor

Indicador	Japa	Jest	Vitest
Tempo médio de parede, Linux (ms) ↓	976,7	629,5	408,7
Tempo médio de parede, Windows (ms) ↓	5.238,3	1.019,8	794,0
Desvio-padrão, Linux / Windows (ms) ↓	66,6 / 267,3	54,9 / 27,1	45,0 / 23,3
Tempo interno, Linux / Windows (ms) ↓	38,6 / 55,2	342,8 / 610,8	168,6 / 303,2
Linhas de teste ↓	130	125	123
Linhas de configuração ↓	71	10	15
Passos de configuração ↓	5	2	2
Cobertura de linhas ↑	45,0%	45,0%	37,3%
Cobertura de ramos ↑	72,7%	72,7%	39,2%
Dependências transitivas ↓	366	322	36
Tamanho em disco (MB) ↓	67	63	12
Curva de aprendizado (1–5) ↑	3	4	5
Documentação oficial (1–5) ↑	3	5	4
Mensagens de erro (1–5) ↑	3	5	4
Integração com IDE e CI/CD (1–5) ↑	3	5	5
Mocking embutido (1–5) ↑	2	5	5
Suporte a TypeScript/ESM (1–5) ↑	4	3	5
Integração com AdonisJS (1–5) ↑	5	2	2
Ecossistema e comunidade (1–5) ↑	2	5	4
Média qualitativa ↑	**3,13**	**4,25**	**4,25**

7. Análise crítica e recomendações

Nenhum framework vence em todos os critérios. O **Vitest** lidera nas métricas objetivas de desempenho e de leveza: menor tempo, menos dependências e configuração mínima, com TypeScript nativo. O **Jest** empata com o Vitest na média qualitativa, graças à documentação, às mensagens de erro e ao ecossistema, mas paga o custo da transformação TypeScript e de uma árvore de dependências grande. O **Japa** é o mais rápido para executar os testes em si e o único integrado ao AdonisJS, mas não traz mocking nem cobertura e carrega o custo de inicializar a aplicação a cada execução.

Cenário	Recomendação	Justificativa
Projeto legado corporativo	**Jest**	Padrão de mercado, documentação extensa, suporte nativo em IDEs e ampla base de conhecimento na equipe. A estabilidade pesa mais que a velocidade.
Startup ágil / projeto TypeScript moderno	**Vitest**	Menor tempo de feedback, configuração em 2 passos, TS/ESM nativos e API compatível com Jest (baixa barreira de migração).
Disciplina introdutória	**Vitest** (ou Jest)	Pouca configuração e mensagens de erro claras; a API `expect` é a mesma do Jest, então o aprendizado é transferível.
Projeto com alta cobertura exigida	**Vitest** ou **Jest**	Cobertura embutida e mocking de módulos facilitam isolar ramos difíceis. A contagem por AST do Vitest é mais rigorosa (não infla o percentual).
Aplicação AdonisJS com testes de integração/HTTP	**Japa**	É o runner oficial: sobe o IoC, usa os models reais, oferece cliente HTTP e transações de banco por teste. Pode ser combinado com Vitest/Jest para testes puramente unitários.

8. Limitações do estudo

- **Amostra pequena:** 12 testes sobre dois serviços. Em suítes grandes, o custo fixo de inicialização (decisivo para o Japa) se dilui, e o paralelismo entre arquivos (Jest e Vitest) passa a pesar.
- **Dois ambientes, sem controle de hardware:** o tempo foi medido em uma VM/contêiner Linux em nuvem e em um computador pessoal com Windows. Os valores absolutos não são comparáveis entre os ambientes; a ordem relativa, que se repetiu nos dois, é o resultado mais confiável. A cobertura de Japa e Jest foi medida apenas no Linux.
- **Cobertura não comparável diretamente:** o Vitest conta linhas por AST, enquanto Japa e Jest convertem a cobertura linha a linha. A diferença percentual vem da ferramenta, não dos testes.
- **Estratégias de dublê diferentes:** o Japa substitui métodos dos models reais (Sinon), enquanto Jest e Vitest substituem o módulo inteiro. Isso é idiomático em cada um, mas faz o Japa carregar mais código.
- **Viés de familiaridade e subjetividade:** as notas qualitativas refletem a experiência desta implementação e podem ser influenciadas pela familiaridade prévia com a API `expect` do Jest.
- **Versões:** o Japa 2 é a versão compatível com o AdonisJS 5; versões mais novas (Japa 3/4, AdonisJS 6) trazem melhorias que não foram avaliadas.

9. Conclusão

Para testes unitários isolados da *girls-chat-api*, o **Vitest** apresentou o melhor equilíbrio: foi o mais rápido nos dois ambientes medidos (no Linux, 2,4 vezes mais rápido que o Japa e 1,5 vezes mais rápido que o Jest; no Windows, 6,6 e 1,3 vezes, respectivamente), exige a menor configuração e empata com o Jest na avaliação qualitativa (4,25). O **Jest** continua a escolha mais segura em

contextos corporativos pela maturidade e pelo ecossistema. O **Japa** se justifica quando os testes precisam da aplicação Adonis inicializada, como nos testes de integração e HTTP já existentes no projeto. A recomendação para o projeto é usar **Vitest para a camada unitária** e manter o **Japa para os testes de integração/funcionais**.

Referências

- Japa: documentação oficial, japa.dev (v2).
- Jest: documentação oficial, jestjs.io.
- Vitest: documentação oficial, vitest.dev.
- AdonisJS 5: documentação de testes, docs.adonisjs.com (v5).
- Sinon.JS: sinonjs.org. c8: github.com/bcoe/c8.
- MARTIN, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008 (princípios FIRST).
- MESZAROS, G. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley, 2007 (dublês de teste).

Apêndice A: mensagens de erro capturadas

Duas falhas foram introduzidas de propósito em uma cópia temporária dos testes: (1) a mensagem de exceção esperada foi trocada para "Chat duplicado" e (2) o nome esperado nos argumentos de `User.create` foi trocado para "Ana Maria".

Jest

```
FAIL tests/frameworks/jest/zz_falha.spec.ts
  ● ChatsService › falha ao criar chat duplicado entre os mesmos usuários

    expect(received).rejects.toThrow(expected)

    Expected substring: "Chat duplicado"
    Received message:   "Erro ao criar o chat: Chat já existe entre esses usuários"

          133 |             return newChat;
          134 |         } catch (error) {
        > 135 |             throw new Error("Erro ao criar o chat: " + error.message);
              |                   ^
          136 |         }
          137 |     }
          138 |

      at ChatsService.createChat (app/services/ChatsService.ts:135:19)
      at Object.<anonymous> (tests/frameworks/jest/zz_falha.spec.ts:70:5)
      at Object.toThrow (node_modules/expect/build/index.js:2155:20)
      at Object.<anonymous> (tests/frameworks/jest/zz_falha.spec.ts:70:56)

  ● UserService › cria usuário armazenando a senha com hash MD5

    expect(jest.fn()).toHaveBeenCalledWith(...expected)

    - Expected
    + Received

      Object {
        "email": "ana@test.com",
    -   "name": "Ana Maria",
    +   "name": "Ana",
        "password": "e7d80ffeefa212b7c5c55700e4f7193e",
      },

    Number of calls: 1

      133 |
      134 |     expect(user.id).toBe(1)
    > 135 |     expect(UserMock.create).toHaveBeenCalledWith({
          |                             ^
      136 |       email: 'ana@test.com',
      137 |       password: md5('senha123'),
      138 |       name: 'Ana Maria',

      at Object.<anonymous> (tests/frameworks/jest/zz_falha.spec.ts:135:29)

Test Suites: 1 failed, 1 total
Tests:       2 failed, 10 passed, 12 total
Snapshots:   0 total
Time:        0.436 s
Ran all test suites matching zz_falha.
```

Vitest

```
 RUN  v5.0.5 /home/user/girls-chat-api

 ❯ tests/frameworks/vitest/zz_falha.spec.ts (12 tests | 2 failed) 26ms
   ❯ ChatsService (9)
     × falha ao criar chat duplicado entre os mesmos usuários 5ms
   ❯ UserService (3)
     × cria usuário armazenando a senha com hash MD5 3ms

⎯⎯⎯⎯⎯⎯ Failed Tests 2 ⎯⎯⎯⎯⎯⎯

 FAIL  tests/frameworks/vitest/zz_falha.spec.ts > ChatsService > falha ao criar chat duplicado entre
os mesmos usuários
AssertionError: expected [Function] to throw error including 'Chat duplicado' but got 'Erro ao criar
o chat: Chat já existe …'

Expected: "Chat duplicado"
Received: "Erro ao criar o chat: Chat já existe entre esses usuários"

 ❯ tests/frameworks/vitest/zz_falha.spec.ts:68:47
     66|     ChatMock.find.mockResolvedValue(fakeChat(10, 1))
     67|
     68|     await expect(service.createChat('1', '2')).rejects.toThrow('Chat d…
       |                                                 ^
     69|     expect(ChatMock.create).not.toHaveBeenCalled()
     70|   })

⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯[1/2]⎯

 FAIL  tests/frameworks/vitest/zz_falha.spec.ts > UserService > cria usuário armazenando a senha com
hash MD5
AssertionError: expected "vi.fn()" to be called with arguments: [ { email: 'ana@test.com', …(2) } ]

Received:

  1st vi.fn() call:

  [
    {
      "email": "ana@test.com",
-     "name": "Ana Maria",
+     "name": "Ana",
      "password": "e7d80ffeefa212b7c5c55700e4f7193e",
    },
  ]


Number of calls: 1

 ❯ tests/frameworks/vitest/zz_falha.spec.ts:133:29
    131|
    132|     expect(user.id).toBe(1)
    133|     expect(UserMock.create).toHaveBeenCalledWith({
       |                             ^
    134|       email: 'ana@test.com',
    135|       password: md5('senha123'),

⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯[2/2]⎯


 Test Files  1 failed (1)
      Tests  2 failed | 10 passed (12)
   Start at  18:22:54
   Duration  196ms (transform 47%, import 26%, tests 25%, worker 3%)
```

Japa

```
[ info ]  running tests...

frameworks / ChatsService (tests/frameworks/japa/zz_falha.spec.ts)
  ✖ falha ao criar chat duplicado entre os mesmos usuários (3ms)

frameworks / UserService (tests/frameworks/japa/zz_falha.spec.ts)
  ✖ cria usuário armazenando a senha com hash MD5 (2ms)

 FAILED

Tests   : 10 passed, 2 failed (12)
Time    : 46ms


✖ falha ao criar chat duplicado entre os mesmos usuários

  Assertion Error: expected [Function] to throw error matching /Chat duplicado/ but got 'Erro ao
criar o chat: Chat já existe …'

  Comparing two different types of values. Expected regexp but received string.

   at Object.executor tests/frameworks/japa/zz_falha.spec.ts:71
   66|      stubs.userFind.callsFake(async (id: any) => fakeUser(id) as any)
   67|      stubs.participantQuery.onFirstCall().returns(fakeQuery([fakeParticipant(10, 1)]))
   68|      stubs.participantQuery.onSecondCall().returns(fakeQuery([fakeParticipant(10, 2)]))
   69|      stubs.chatFind.resolves(fakeChat(10, 1))
   70|
 ❯ 71|      await assert.rejects(() => service.createChat('1', '2'), /Chat duplicado/)
   72|      assert.isFalse(stubs.chatCreate.called)
   73|    })
   74|
   75|    test('encontra o chat em comum entre dois usuários', async ({ assert }) => {
   76|      stubs.participantQuery


✖ cria usuário armazenando a senha com hash MD5

  Assertion Error: expected { email: 'ana@test.com', …(2) } to deeply equal { email: 'ana@test.com',
…(2) }

- Expected  - 1
+ Received  + 1

  Object {
    "email": "ana@test.com",
-   "name": "Ana Maria",
+   "name": "Ana",
    "password": "e7d80ffeefa212b7c5c55700e4f7193e",
  }

   at Object.executor tests/frameworks/japa/zz_falha.spec.ts:143
   138|      stubs.userCreate.callsFake(async (data: any) => ({ id: 1, ...data }))
   139|
   140|      const user = await service.createUser('ana@test.com', 'senha123', 'Ana')
   141|
   142|      assert.equal(user.id, 1)
 ❯ 143|      assert.deepEqual(stubs.userCreate.firstCall.args[0], {
   144|        email: 'ana@test.com',
   145|        password: md5('senha123'),
   146|        name: 'Ana Maria',
   147|      })
   148|    })
```